

Tópico 1

Nomes: Luciano Gonçalves Padilha Junior, Emanuel Cardoso da Silva

Bryan Henrycky da Cruz Dombroski

Turma: V2

Instituição: Senai Sul Joinville-SC

Disciplina: Testes de Sistemas

1. Testes não Funcionais(Performance,Carga,Estresse)

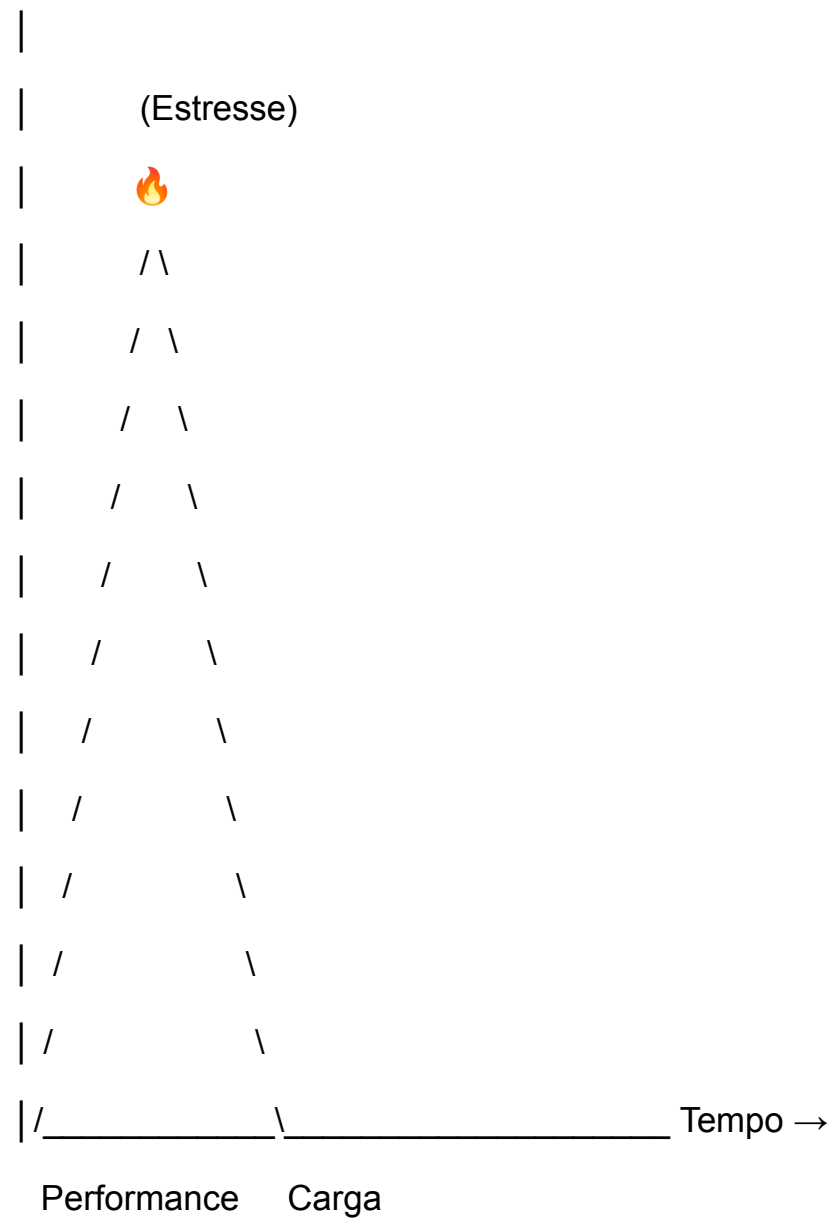
Introdução

Os testes não funcionais são essenciais porque garantem que um sistema não apenas funcione corretamente, mas também ofereça um bom desempenho em situações reais de uso. Em ambientes com muitos usuários simultâneos, como e-commerces ou aplicativos bancários, falhas de desempenho podem causar lentidão, travamentos ou até a indisponibilidade total do sistema. Por isso, avaliar aspectos como tempo de resposta, estabilidade e capacidade de processamento é fundamental para garantir a qualidade do software.

Na prática, esses testes são realizados simulando diferentes níveis de uso do sistema. O teste de performance mede a velocidade de resposta, o teste de carga verifica o comportamento sob volume esperado de usuários, e o teste de estresse avalia o limite máximo do sistema até sua falha. Ferramentas como JMeter, Gatling e K6 são utilizadas para simular acessos simultâneos, permitindo identificar gargalos (bottlenecks) na arquitetura, como problemas no banco de dados, servidor ou rede.

Visualização Gráfica

Usuários Simultâneos ↑



Legenda:

✓ Performance → tempo de resposta normal

✓ Carga → limite esperado

🔥 Estresse → ponto de falha (queda do sistema)

5 Perguntas Desafiadoras

1. Qual a principal diferença entre testes de carga e testes de estresse em relação ao comportamento do sistema?
2. O que é um gargalo de desempenho (bottleneck) e como ele impacta a experiência do usuário?
3. Como as métricas de throughput e latência ajudam a avaliar a performance de um sistema?
4. Por que um sistema pode funcionar bem com poucos usuários, mas falhar sob alta demanda?
5. Qual a importância de utilizar ferramentas automatizadas em testes não funcionais?

3 Exemplos de Teste

♦ Teste 1

- **Cenário:** 1.000 usuários acessando um site de e-commerce ao mesmo tempo
 - **Como Testar:** Utilizar JMeter para simular múltiplos acessos simultâneos
 - **Objetivo:** Verificar se o sistema mantém tempo de resposta aceitável sob carga normal
-

♦ Teste 2

- **Cenário:** Aumento gradual de usuários até o sistema parar de responder
 - **Como Testar:** Usar K6 para aumentar progressivamente o número de requisições
 - **Objetivo:** Identificar o limite máximo do sistema e o ponto de falha
-

♦ Teste 3

- **Cenário:** Sistema com lentidão ao acessar o banco de dados
- **Como Testar:** Monitorar consultas ao banco durante teste de carga
- **Objetivo:** Detectar gargalos no banco de dados (bottleneck)

Tópico 2

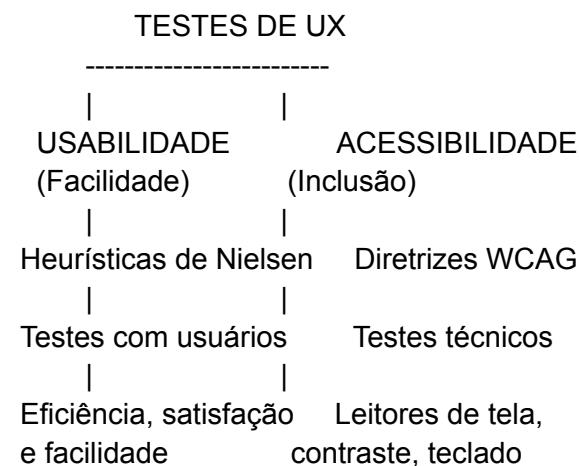
2. Testes de Usabilidade e Acessibilidade Digital (UX)

A. Introdução (Por quê e Como)

A preocupação com usabilidade e acessibilidade em sistemas digitais existe porque nem todos os usuários interagem com a tecnologia da mesma forma. Interfaces mal projetadas podem causar frustração, erros e até exclusão de pessoas com deficiências. A usabilidade busca garantir que o sistema seja fácil de aprender, eficiente e agradável, enquanto a acessibilidade assegura que qualquer pessoa — incluindo aquelas com limitações visuais, motoras ou cognitivas — consiga utilizar o software. Além disso, existem normas e diretrizes internacionais, como o World Wide Web Consortium, que definem padrões obrigatórios no mercado.

O processo de teste envolve diferentes abordagens. A usabilidade pode ser avaliada com base nas **10 heurísticas de Jakob Nielsen**, além de testes com usuários reais. Já a acessibilidade é guiada pelas diretrizes Web Content Accessibility Guidelines, que fornecem critérios técnicos verificáveis (como contraste de cores e navegação por teclado). Ferramentas como leitores de tela — por exemplo, NVDA e VoiceOver — ajudam a validar se o sistema é realmente acessível na prática.

B. Visualização Gráfica do Tópico



C. 5 Perguntas Desafiadoras

1. Qual a principal diferença entre testes heurísticos e testes com usuários reais, e quando usar cada um?
2. Por que as diretrizes WCAG são consideradas mais objetivas que testes de usabilidade tradicionais?

3. Como garantir que um sistema seja acessível sem depender apenas de ferramentas automáticas?
 4. Quais problemas podem surgir ao ignorar acessibilidade em um sistema comercial?
 5. De que forma leitores de tela ajudam a identificar falhas que não são visíveis para usuários comuns?
-

D. Exemplos de Testes

1. Teste de Contraste de Cores (Acessibilidade)

- **Cenário:** Um site com textos em fundo colorido.
 - **Como Testar:** Utilizar ferramentas de contraste ou verificar manualmente conforme critérios WCAG.
 - **Objetivo:** Garantir que usuários com baixa visão consigam ler o conteúdo sem dificuldade.
-

2. Teste de Navegação por Teclado (Acessibilidade)

- **Cenário:** Usuário que não utiliza mouse.
 - **Como Testar:** Navegar pelo sistema apenas com a tecla TAB e ENTER.
 - **Objetivo:** Verificar se todos os elementos são acessíveis sem uso de mouse.
-

3. Teste de Eficiência de Tarefa (Usabilidade)

- **Cenário:** Usuário precisa fazer cadastro em um sistema.
- **Como Testar:** Observar usuários reais realizando a tarefa e medir tempo, erros e dificuldades.
- **Objetivo:** Avaliar se o processo é simples, rápido e intuitivo.

Tópico 3

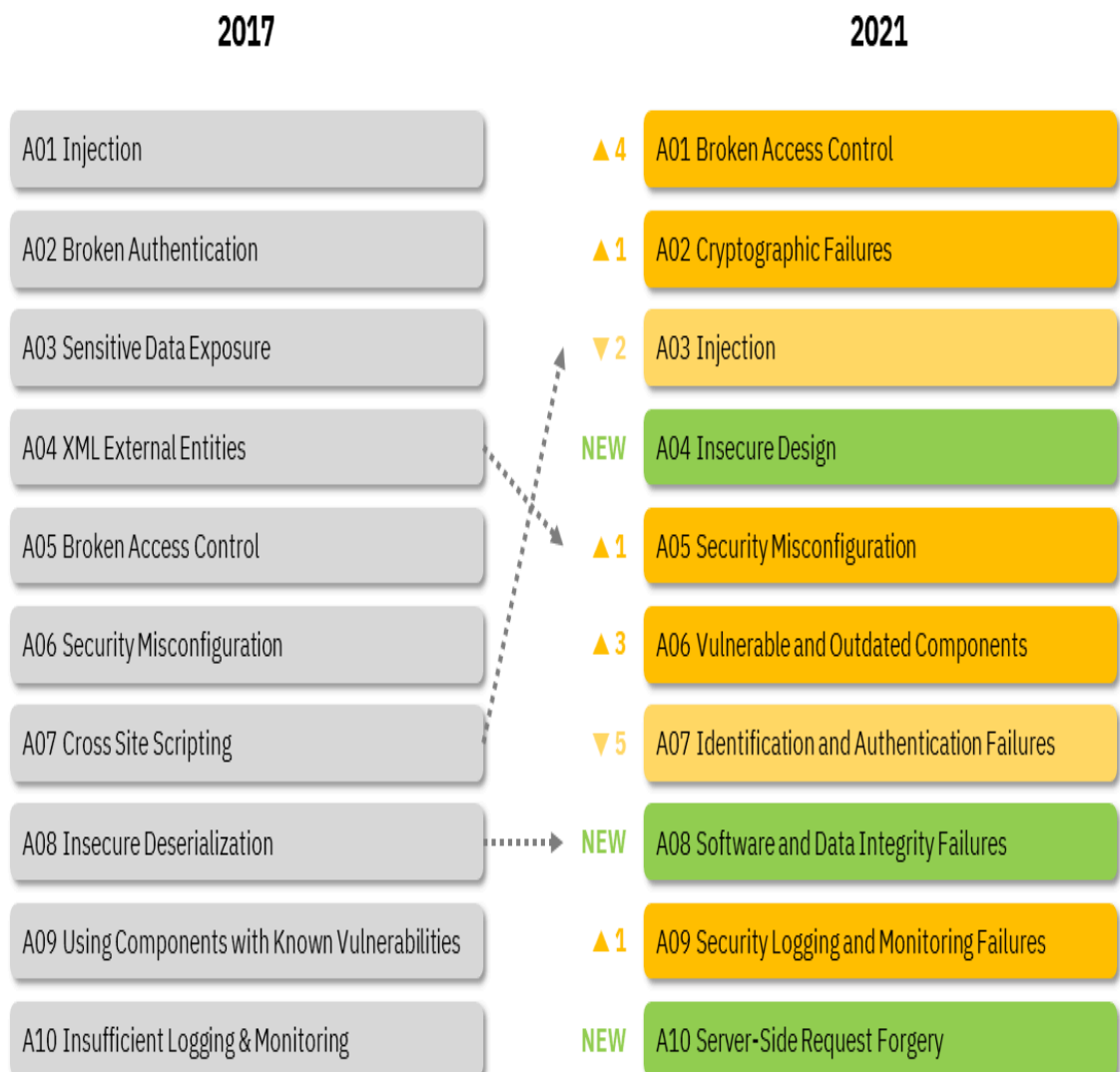
3. Fundamentos de Segurança da Informação nos Testes

A. Introdução: Porquê e Como

A segurança da informação é parte fundamental do QA moderno, pois protege dados e evita prejuízos financeiros, danos à reputação e problemas legais (como a LGPD).

A aplicação prática envolve **Security by Design**, uso do **OWASP Top 10** e testes como SAST, DAST e Pentest, garantindo que o sistema funcione corretamente e resista a ataques.

B. Visualização





SAST vs. DAST

Static application security testing (SAST) and dynamic application security testing (DAST) are both methods of testing for security vulnerabilities, but they're used very differently.

Here are some key differences between the two:

White box security testing

- The tester has access to the underlying framework, design, and implementation.
- The application is tested from the inside out.
- This type of testing represents the developer approach.



Black box security testing

- The tester has no knowledge of the technologies or frameworks that the application is built on.
- The application is tested from the outside in.
- This type of testing represents the hacker approach.

Requires source code

- SAST doesn't require a deployed application.
- It analyzes the source code or binary without executing the application.



Requires a running application

- DAST doesn't require source code or binaries.
- It analyzes by executing the application.

Finds vulnerabilities earlier in the SDLC

- The scan can be executed as soon as code is deemed feature-complete



Finds vulnerabilities toward the end of the SDLC

- Vulnerabilities can be discovered after the development cycle is complete.



Less expensive to fix vulnerabilities

- Since vulnerabilities are found earlier in the SDLC, it's easier and faster to remediate them.
- Findings can often be fixed before the code enters the QA cycle.



More expensive to fix vulnerabilities

- Since vulnerabilities are found toward the end of the SDLC, remediation often gets pushed into the next cycle.
- Critical vulnerabilities may be fixed as an emergency release.



Can't discover run-time and environment-related issues

- Since the tool scans static code, it can't discover run-time vulnerabilities.



Can discover run-time and environment-related issues

- Since the tool uses dynamic analysis on an application, it is able to find run-time vulnerabilities.



Typically supports all kinds of software

- Examples include web applications, web services, and thick clients.



Typically scans only apps like web applications and web services

- DAST is not useful for other types of software.



SAST and DAST techniques complement each other.

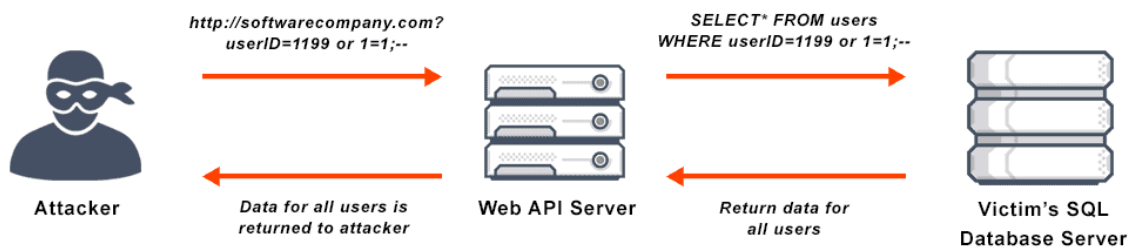


Both need to be carried out for comprehensive testing.



To learn how to create a comprehensive software security testing program, visit www.BlackDuck.com/software

3. Fluxo de Validação de Input (SQL Injection e XSS)



C. 5 Perguntas-Chave

- **Autenticação vs Autorização:** Como diferenciar testes de identidade vs permissões em APIs?
- **Injeção:** Como entradas não validadas permitem Command Injection?
- **Configuração:** Por que erros detalhados em produção são perigosos?
- **Criptografia:** Qual o impacto da ausência de Forward Secrecy?
- **CI/CD:** Quando executar SAST e DAST sem afetar a agilidade?

D. 3 Exemplos de Teste

- **SQL Injection:** Inserir comandos maliciosos → validar uso de queries parametrizadas.
- **XSS (armazenado):** Inserir scripts → verificar encoding de saída.
- **IDOR:** Alterar IDs na URL → validar controle de acesso no servidor.

Exercícios

1. Diferença prática: performance vs carga vs estresse

Não é só “nomes diferentes”, cada um responde a uma pergunta específica:

- Teste de Performance → “*Está rápido o suficiente?*”
Mede tempo de resposta, latência e fluidez sob condições normais.
- Teste de Carga → “*Aguenta o volume esperado?*”
Simula usuários reais esperados (ex: 1.000 usuários simultâneos) e observa estabilidade.
- Teste de Estresse → “*Onde quebra?*”
Vai além do limite até o sistema falhar — identifica ponto de ruptura e comportamento sob falha.

Resumo prático:

- Performance = velocidade
 - Carga = capacidade planejada
 - Estresse = limite máximo + falha
-

2. Throughput e Latência

- Throughput (Taxa de transferência)
→ Quantidade de requisições processadas por segundo
Ex: 500 req/s
- Latência (Tempo de resposta)
→ Tempo que uma requisição leva do início ao fim
Ex: 200 ms

Como ferramentas medem isso:

Ferramentas como Apache JMeter, Gatling e k6 fazem:

- Simulação de usuários virtuais
- Envio de requisições HTTP
- Coleta automática de métricas

Exemplo prático:

- Latência → tempo entre request e response
- Throughput → número de requests completadas por segundo

3. Por que encontrar gargalo é o objetivo real

Rodar teste sem análise é basicamente inútil.

O valor real está em responder:

- Onde está o problema?
 - Banco? (queries lentas)
 - Backend? (CPU alta)
 - Rede? (timeout)

Sem identificar o bottleneck:

- Você só sabe *que está lento*, não *por quê*

Com identificação:

- Você sabe exatamente *onde otimizar*
-

Testes de Usabilidade e Acessibilidade (UX)

4. 10 Heurísticas de Nielsen

Criadas por Jakob Nielsen, são princípios para avaliar interfaces.

Algumas das mais importantes:

- Visibilidade do status do sistema
- Correspondência com o mundo real
- Controle e liberdade do usuário
- Consistência e padrões
- Prevenção de erros

Elas funcionam como um checklist mental para UX:

“Isso faz sentido para um usuário comum?”

5. Usuários reais vs teste heurístico

Teste com usuários reais

- correto: Mostra problemas reais
- correto: Revela comportamento inesperado
- errado: Mais caro e demorado

Teste heurístico

- correto: Rápido e barato
- correto: Pode ser feito por especialistas
- errado: Pode ignorar problemas reais de uso

Melhor prática:

- Começar com heurístico
 - Validar com usuários reais
-

6. WCAG

O padrão World Wide Web Consortium define o WCAG.

Ele transforma acessibilidade em critérios objetivos:

Exemplos:

- Imagens têm texto alternativo? → PASS/FAIL
- Contraste está adequado? → PASS/FAIL
- Navegação por teclado funciona? → PASS/FAIL

Ou seja:

- Sai do subjetivo (“parece acessível”)
 - Vai para o mensurável (“está conforme?”)
-

7. Uso prático de leitores de tela

Ferramentas como:

- NVDA
- VoiceOver

Devem ser usadas assim:

- Navegar sem olhar a tela
- Usar apenas teclado
- Ouvir se:
 - Botões têm descrição
 - Fluxo faz sentido
 - Ordem de leitura está correta

Se o leitor “se perde”, o usuário também vai.

Segurança da Informação nos Testes

8. OWASP e OWASP Top 10

A OWASP é referência global em segurança.

O OWASP Top 10:

- Lista as vulnerabilidades mais críticas
- Baseado em dados reais de ataques

Exemplos:

- Injection
- Broken Authentication
- XSS

Por que é padrão de mercado?

- Atualizado constantemente
 - Baseado em evidência real
 - Amplamente adotado por empresas
-

9. Falta de validação de input

Se o sistema aceita qualquer entrada:

Pode acontecer:

- SQL Injection

' OR 1=1 --

→ acesso indevido ao banco

- XSS (Cross-Site Scripting)

<script>alert('hack')</script>

→ execução de código no navegador

Causa raiz:

- Entrada não validada/sanitizada
-

10. SAST vs DAST vs Pentest

SAST (Static Application Security Testing)

- Analisa código fonte
- Antes da execução
- Ex: detectar vulnerabilidades no código

DAST (Dynamic Application Security Testing)

- Testa sistema rodando
- Simula ataques externos

Pentest (Manual)

- Feito por especialistas
- Exploração real de falhas
- Criativo, não automatizado

Diferença essencial:

- SAST → preventivo (código)
- DAST → reativo (execução)
- Pentest → investigativo profundo